

DATABASE MANAGEMENT SYSTEMS

(TAE-2 : Mini Project Documentation)

Team Members :

Sr.	Name	Grade	Roll No.
1	Chandana Kunjir	T.Y. B.Tech CSE (Cyber Security)	TB1-CYSEC-34
2	Harsh Jediya	T.Y. B.Tech C.E.	TB1-CE-12

PROJECT TITLE : Cyber Incident Reporting and Tracking System

1. Introduction :

The Cyber Incident Reporting and Tracking System (branded as CyberGuard) is a web-based DBMS project that provides a simple and organized way to report, store, manage and track cybersecurity incidents. It is suitable for organizations, colleges and small institutions where users need to report cyber-related problems and administrators need to monitor their progress.

- Cybersecurity incidents such as phishing emails, malware infections, hacking, suspicious websites and unauthorized access must be reported and monitored properly.
- The system stores all incident details in a centralized MySQL database.
- Users can submit incidents; administrators can view them and update their status.
- Every status change is recorded, so a full history of each incident is maintained.
- Developed using MySQL, Python, Flask, HTML and CSS.
- Demonstrates practical implementation of DBMS concepts in a real-world scenario.

2. Problem & Proposed Solution :

Problem in Existing (Manual) System	Proposed Solution
Incident details are scattered across emails, calls and paper records	Centralized MySQL database stores all incident data
No standard format for reporting an incident	Structured web form with title, type, description, severity, location and date
Difficult to know the current status of an incident	Status tracking: Reported → Investigating → Resolved
No record of who changed what and when	incident_updates table keeps the history of every status change
Manual record keeping is slow and error-prone	Automated storage and retrieval through Python Flask

Problem in Existing (Manual) System	Proposed Solution
Lack of simple tools for small institutions	Lightweight, open-source, easy-to-use dashboard

3. Objectives & Future Scope :

Objectives

- Provide an easy method for reporting cyber incidents.
- Store incident information securely in a database.
- Allow users to view reported incidents.
- Track the current status of incidents.
- Allow administrators to update incident status.
- Maintain a history of status changes.
- Demonstrate practical implementation of DBMS concepts.
- Connect a MySQL database with a Python Flask web application.

Future Scope

- Secure login and authentication with password hashing.
- Role-based access control for users and administrators.
- Email / SMS notifications, including automatic alerts when status changes.
- File / evidence uploading with incident reports.
- Search and filtering of incidents.
- Dynamic dashboard statistics and advanced incident reports.
- Better security and audit logging.

4. Comparative Analysis :

Existing Approach	Proposed Approach
- Manual incident recording (paper, email, phone)	- Automated web-based incident reporting
- Data is scattered and hard to organize	- Centralized, structured MySQL database
- Status tracking done informally	- Clear status workflow with tracked history
- Uses expensive enterprise ticketing tools	- Uses open-source tools (MySQL, Flask)
- Complex setup and operation	- Simple setup on a local machine
- Limited accessibility for small organizations	- Suitable for colleges and small institutions
Existing System	Proposed Project

5. Tools & Technologies Used :

Technology	Purpose
MySQL	Database management – stores users, incidents and update history
MySQL Workbench	Creating and managing the database and tables
Python	Backend programming
Flask	Web application framework – routes and request handling
MySQL Connector	Connecting Python with the MySQL database
HTML	Structure of the web pages
CSS	Website design and dark cybersecurity-themed styling

6. Database Design :

The database is named **cyber_incident_db** and contains three main tables connected through foreign keys.

Users Table

Stores information about system users.

Field	Description
user_id	Primary key – unique ID of each user
name	Name of the user
email	Email address of the user
password	Password (demonstration data)
role	Role of the user (user / administrator)

Incidents Table

Stores information about cybersecurity incidents.

Field	Description
incident_id	Primary key – unique ID (AUTO_INCREMENT)
user_id	Foreign key – references the user who reported the incident
title	Short title of the incident

Field	Description
incident_type	Phishing, Virus, Malware, Hacking, etc.
description	Detailed description of the incident
severity	Low / Medium / High
location	Where the incident occurred
incident_date	Date of the incident
status	Reported / Investigating / Resolved

Incident Updates Table

Stores the history of changes made to an incident.

Field	Description
update_id	Primary key – unique ID of each update
incident_id	Foreign key – references the incident being updated
old_status	Status before the change
new_status	Status after the change
update_description	Note describing the update
update_date	Date and time of the update

7. ER Diagram :

The Entity-Relationship (ER) diagram below shows the three entities of the system, their attributes and the relationships between them.

ER Diagram - Cyber Incident Reporting and Tracking System

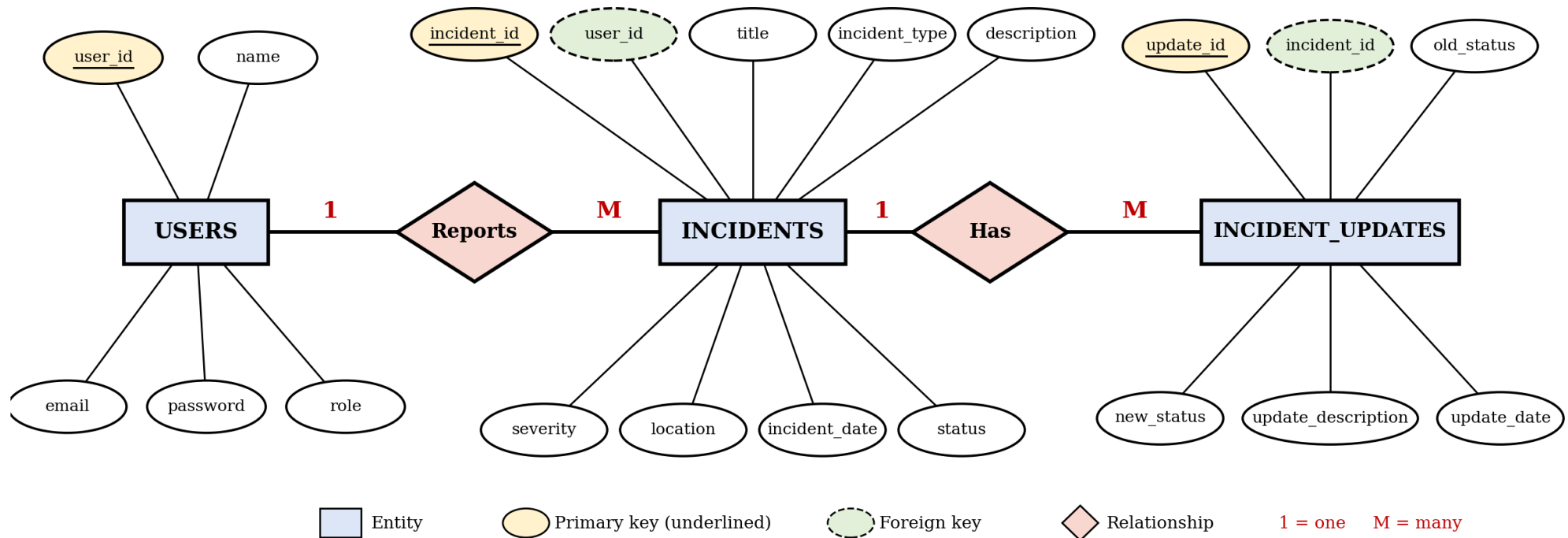


Figure: ER Diagram of the Cyber Incident Reporting and Tracking System

Relationships

- **Reports (Users 1 : M Incidents)** – one user can report many incidents; linked through user_id.
- **Has (Incidents 1 : M Incident_Updates)** – one incident can have many status updates; linked through incident_id.

8. Methodology :

Step 1: Database Creation

- Create the cyber_incident_db database in MySQL Workbench.
- Create the users, incidents and incident_updates tables with primary and foreign keys.

Step 2: Backend Development

- Develop the Flask application in Python.
- Connect Flask to MySQL using MySQL Connector.

Step 3: Frontend Development

- Design the dashboard, report form, incident table and admin panel using HTML and CSS.
- Apply a dark cybersecurity theme with status badges.

Step 4: Incident Reporting and Tracking

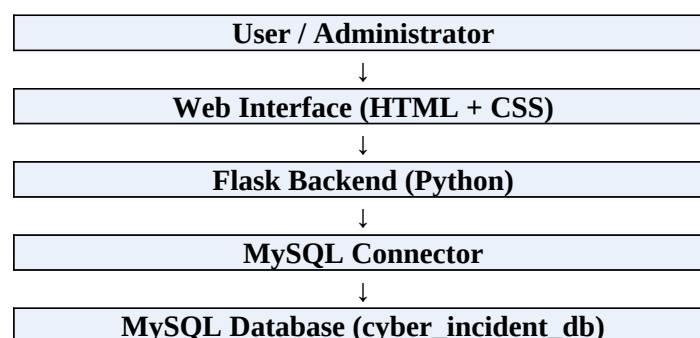
- Store submitted incidents in the database with the initial status “Reported”.
- Let the administrator change the status to “Investigating” and then “Resolved”.

Step 5: Testing

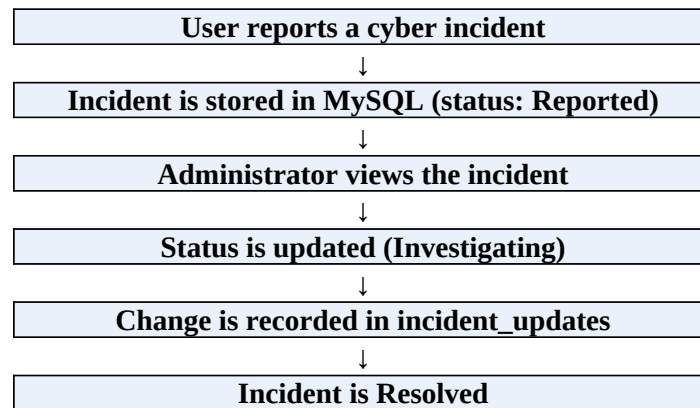
- Test reporting, viewing and status updating with sample incidents.
- Verify that the database records match what is shown on the web pages.

9. System Architecture :

The system follows a simple three-layer architecture: a web interface for users, a Flask backend that processes requests, and a MySQL database that stores the data.



Working of the System



Status flow: Reported → Investigating → Resolved

10. Implementation :

I. Backend (Python Flask)

- Flask handles the routes: /, /report, /incidents, /admin and /update_status.
- When a user submits an incident, Flask receives the form data and inserts it into MySQL.
- When the incidents page opens, Flask retrieves the records from MySQL and displays them.
- When the administrator changes a status, Flask updates the corresponding record.

II. Main Modules

Module	Function
Dashboard	Overview of the system with quick access to different functions
Report Incident	Form to enter title, type, description, severity, location and date
View Incidents	Displays stored incidents with their status and other details
Admin Panel	Lets the administrator view incidents and change their status
Incident Tracking	Uses the incident_updates table to record status changes

III. User Interface

- CyberGuard branding with a navigation sidebar.
- Forms, tables, buttons and colour-coded status badges (Reported, Investigating, Resolved).
- Dark cybersecurity-themed design that is simple and professional.

IV. DBMS Concepts Demonstrated

Database and table creation, primary keys, foreign keys, AUTO_INCREMENT, insert / update / delete operations, SELECT queries, WHERE, ORDER BY, LIKE, aggregate functions, GROUP BY, HAVING, INNER JOIN, date functions, string functions and relationships between tables.

V. Sample SQL Queries

INNER JOIN – show which user reported each incident:

```
SELECT i.incident_id, i.title, u.name, i.status
FROM incidents i
INNER JOIN users u ON i.user_id = u.user_id;
```

GROUP BY / HAVING – incident types reported more than once:

```
SELECT incident_type, COUNT(*) AS total
FROM incidents
GROUP BY incident_type
HAVING COUNT(*) > 1;
```

LIKE and ORDER BY – search incidents by title:

```
SELECT * FROM incidents
WHERE title LIKE '%Email%'
ORDER BY incident_date DESC;
```

VI. Testing & Validation

- Reported sample incidents (phishing, virus, hacking, malware) through the web form.
- Verified they appear on the Incident Records page with the correct details.
- Updated statuses from the Admin Panel and confirmed the changes in the database.

11. Output :

The following screenshots show the working application running on the local server (127.0.0.1:5000).

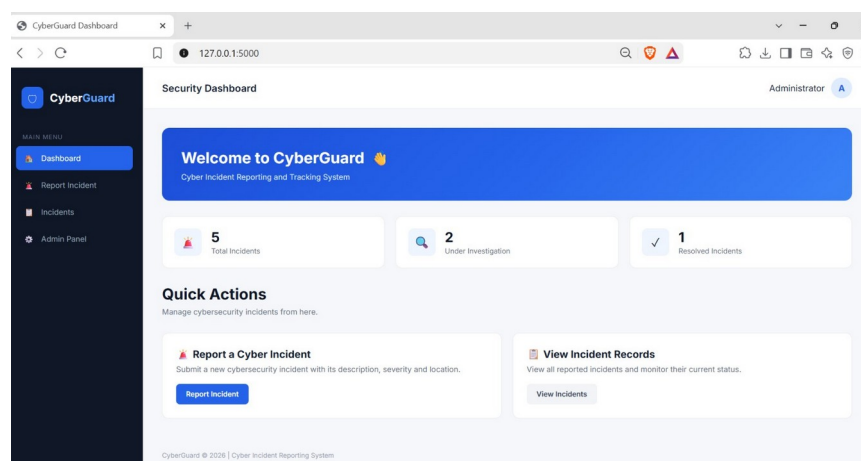


Figure 1: Security Dashboard

Report Incident

Enter the details of the cybersecurity incident.

Incident Title
Example: Suspicious Email

Incident Type
Phishing

Description
Describe the incident...

Severity
Low

Location
Example: Pune

Incident Date
dd-mm-yyyy

Figure 2: Report Incident Page

Incident Records

View and track all reported cybersecurity incidents.

ID	TITLE	TYPE	SEVERITY	LOCATION	DATE	STATUS
#1	Suspicious Email	Phishing	High	Pune	2026-09-29	Resolved
#2	Suspicious Software	Virus	High	Pune	2026-09-30	Investigating
#5	Social Media Account Hack	Hacking	High	Mumbai	2026-09-28	Reported
#10	Fake Login Website	Phishing	High	Pune	2026-09-26	Resolved
#11	Suspicious Website	Phishing	High	Pune	2026-09-29	Reported
#12	Suspicious email	Phishing	High	Pune	2026-09-29	Reported
#13	Malware	Malware	High	Mumbai	2026-09-29	Reported
#14	Cyber Attack	Hacking	High	Pune	2026-09-17	Reported

Figure 3: Incident Records Page

Admin Panel

Manage and update incident statuses.

ID	TITLE	TYPE	SEVERITY	STATUS	UPDATE STATUS
#1	Suspicious Email	Phishing	High	Resolved	Reported Update
#2	Suspicious Software	Virus	High	Investigating	Reported Update
#5	Social Media Account Hack	Hacking	High	Reported	Reported Update
#10	Fake Login Website	Phishing	High	Resolved	Reported Update
#11	Suspicious Website	Phishing	High	Reported	Reported Update

Figure 4: Admin Panel (Update Incident Status)

12. Advantages & Limitations :

Advantages

- Easy incident reporting and centralized data storage.
- Faster incident tracking and easy status updating.
- Reduced manual record keeping.

- Simple, user-friendly interface.
- Demonstrates real-world use of DBMS.

Limitations

- Full user authentication is not implemented.
- The admin panel is simple and has no advanced access control.
- Passwords are stored as demonstration data and are not securely hashed.
- No email or SMS notifications.
- No AI-based threat detection.
- Dashboard statistics are basic / hard-coded rather than fully dynamic.

13. Conclusion :

The Cyber Incident Reporting and Tracking System successfully demonstrates how a database-driven web application can be developed using MySQL and Python Flask. It provides a structured method for reporting cybersecurity incidents, storing their information, viewing incidents and tracking their progress through the Reported → Investigating → Resolved workflow. The project combines theoretical database concepts such as tables, keys, relationships and SQL queries with a practical web interface, and has clear scope for future improvements such as secure authentication, notifications and dynamic reporting.